

游戏代码架构与设计模式

一、游戏架构模式总览

游戏开发中常用的架构模式决定了代码的可维护性和性能。核心模式包括：ECS(Entity Component System)、组件式(Component-Based)、状态机(State Machine)、对象池(Object Pool)、事件总线(Event Bus)、命令模式(Command Pattern)。

二、ECS vs 组件式架构

2.1 ECS (Entity Component System)

ECS 将 数据(Component) 与 逻辑(System) 完全分离。Entity 只是一个 ID，Component 存储纯数据，System 对所有包含特定组件的 Entity 执行逻辑。优势：高并发（System 可并行执行）、高灵活性（动态添加/移除组件）、缓存友好。适合：实体数量巨大的游戏（如 Vampire Survivors 类）。

2.2 组件式 (Component-Based)

传统的 GameObject + Component 模式。GameObject 持有多个 Component，每个 Component 有自身的 Update() 方法。优势：直观易理解、Unity/Unreal 原生支持。适合：实体数量适中的游戏（RPG、动作游戏）。

小型Demo推荐使用组件式或更简单的面向对象模式：Hero 类直接包含 name/hp/skills/cooldowns 属性，BattleSystem 类协调战斗流程，避免过度设计。

三、状态机模式 (State Machine)

状态机是管理角色/UI/游戏流程状态的标准方案。每个状态有3个生命周期：OnEnter() - 状态进入时执行一次，OnUpdate() - 每帧/每回合执行，OnExit() - 状态退出时清理。

战斗角色状态机示例：Idle(待机) → Attack(攻击, 播放动画→造成伤害→回Idle) → Hit(受击, 播放僵直→回Idle) → Dead(死亡, 播放死亡动画→移除)。

注意：简单的回合制 Demo 不需要完整状态机。直接在 Fighter.use_skill() 中同步完成「播放动画→计算伤害→扣HP」即可，避免异步状态切换导致的伤害延迟。

四、对象池模式 (Object Pool)

对象池是游戏性能优化的核心模式。频繁创建和销毁对象（子弹、特效、敌人）会导致 GC 压力，造成帧率波动。对象池预创建一批实例，使用时从池中获取(get)，用完后归还(release)。

实现参考：class ObjectPool: 维护一个 _available: list 和一个 _in_use: set。acquire() 从 _available pop，没有可用时创建新实例。release(obj) 重置 obj 状态并放回 _available。

五、事件总线 (Event Bus)

事件总线解耦模块间通信。模块不直接调用对方，而是发布(Publish)事件和订阅(Subscribe)事件。常见事件类型：

ENTITY_DIED(entity_id)、DAMAGE_DEALT(source, target, amount)、LEVEL_COMPLETE、WAVE_START。

实现：EventBus 维护 handlers: dict[str, list[callable]]。emit(event_name, **data) 调用所有已注册的 handler。小型 Demo 不需要事件总线，直接用 BattleSystem 协调即可。

六、游戏主循环

标准游戏主循环(Game Loop): while running: dt = clock.tick(FPS); process_input(dt); update(dt); render()。固定时间步长变体: accumulator += dt; while accumulator >= FIXED_DT: update(FIXED_DT); accumulator -= FIXED_DT。

Demo 代码中，回合制游戏不需要 while True。用 for round_num in range(1, max_rounds+1) 即可，每回合调用 fighter.use_skill() + print状态。塔防用 for wave in range(1, max_waves+1)，每波内部用 for turn in range(wave_duration)。

七、Python 游戏代码最佳实践

1) 类型注解: def use_skill(self, target: Fighter) -> int 2) 数据类: 用 dict 或 dataclass 存储技能配置 3) 单一职责: Plant 只管攻击判定, BattleSystem 管流程协调 4) 数值外置: SKILL_DATA、ZOMBIE_TYPES 等配置用字典常量，不在类内部硬编码 5) 测试友好: 核心逻辑函数纯函数化，方便 pytest 测试。